Tên: Lê Thiên Hạo

MSSV: 089206010393

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from enum import Enum

class point(Enum):
    CORE = "CORE"
    BORDER = "BORDER"
    NOISE = "NOISE"
```

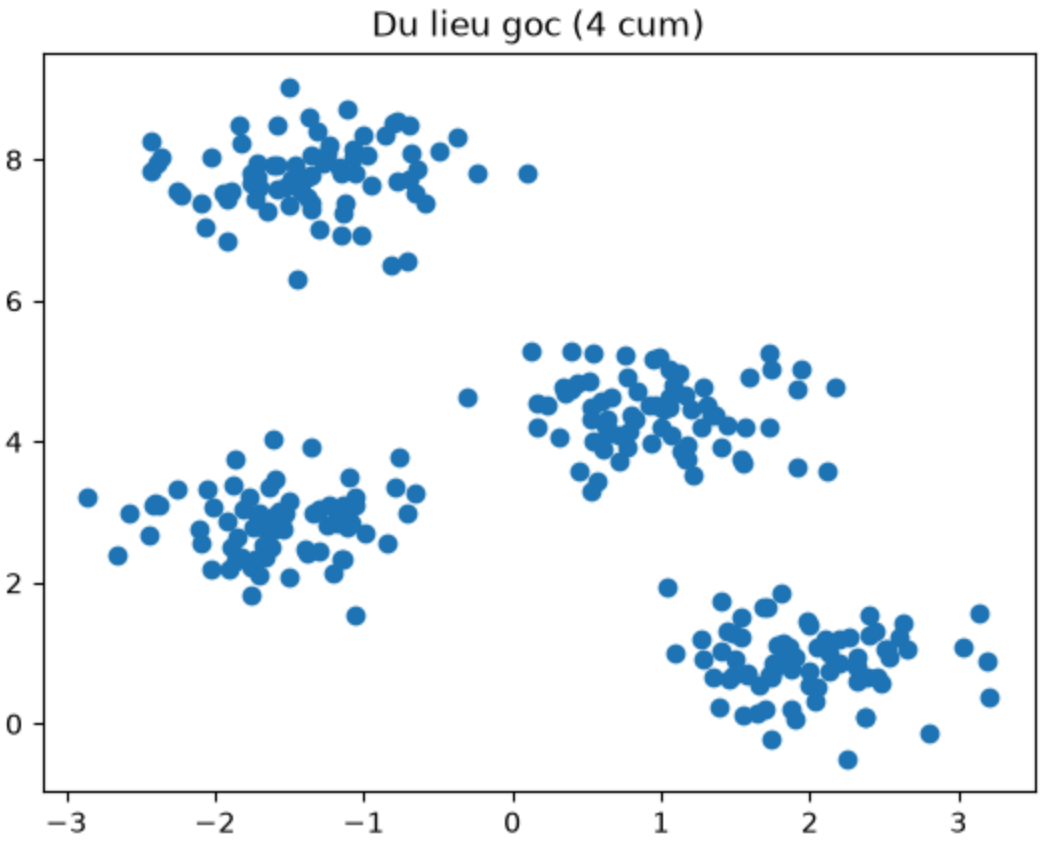
- Density at point `p` : number of points within a circle of radius `Eps`
- Dense Region: a circle of radius `Eps` that contains at least `MinPts` points

A point is a **core point** if it has more than a specified number of points `MinPts` within `Eps` .

- These points belong to a dense region and are at the interior of a cluster.
- Density-connected: consecutive points connected within radius `Eps` .
- **core point**: has at least `MinPts` neighbors within radius `eps`
- **border point**: has fewer than `MinPts` neighbors, but lies within `eps` of a core point
- **noise point**: neither a core point nor a border point

In [2]:
```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=300, centers=4,
                  cluster_std=0.50, random_state=0)
```

In [3]:
```
plt.scatter(X[:, 0], X[:, 1])
plt.title("Du lieu goc (4 cum)")
plt.show()
```



Tính ma trận khoảng cách (ĐÃ SỬA LỖI)

In [4]:
```
distance = []
for p in X:
    temp = []
    for p_ in X:
        # SUA: truoc day la (p_[1] - p_[1]) luon = 0 -> khoang cach chi tinh
        eq = ((p_[0] - p[0]) ** 2 + (p_[1] - p[1]) ** 2) ** 0.5
        temp.append(eq)
    distance.append(temp)

distance = pd.DataFrame(distance)
distance.shape
```

Out[4]: (300, 300)

In [5]:
```
data = np.array(distance)
data.shape
```

Out[5]: (300, 300)

In [6]:
```
distance
```

Out[6]:

	0	1	2	3	4	5	6	7	
0	0.000000	6.057695	3.038272	6.250618	0.415191	2.671672	2.803529	2.456863	6
1	6.057695	0.000000	3.553280	0.471640	6.393783	8.467141	5.675852	4.137939	1
2	3.038272	3.553280	0.000000	3.590625	3.252037	5.048547	4.272533	0.636280	3
3	6.250618	0.471640	3.590625	0.000000	6.569160	8.584600	6.042381	4.201417	0
4	0.415191	6.393783	3.252037	6.569160	0.000000	2.259625	3.164633	2.642258	6
...	...	...	...	...	...	...	...	...	
295	1.054806	6.895924	3.605409	7.043995	0.640691	1.641596	3.765031	2.973301	7
296	1.532475	7.261635	3.887035	7.389347	1.119822	1.213987	4.233061	3.250755	7
297	2.617765	3.544887	0.759533	3.680120	2.904939	4.922279	3.525100	0.822452	3
298	5.943404	1.519573	3.011513	1.156916	6.209828	8.054387	6.264217	3.647791	0
299	2.913373	4.716748	3.611876	5.087706	3.320102	5.509656	0.960644	3.473728	5

300 rows × 300 columns

Bước 1: Phân loại core / border / noise

`EPS` và `MinPts` bên dưới đã được dò thử lại (bản gốc dùng EPS=1.0/2.0 và MinPts=40/10 nhưng vì ma trận khoảng cách trước đó bị sai nên các giá trị đó không còn hợp lý). Với dữ liệu 4 cụm, std=0.5 như trên, `EPS=0.6, MinPts=8` cho kết quả 4 cụm rõ ràng, ít noise.

In [7]:
```
EPS = 0.6
MinPts = 8
```

In [8]:
```
def dbscan_manual(df, eps, min_samples):
    dist = df.values  # ma tran khoang cach NxN
    n = dist.shape[0]

    # Buoc 1: Tim neighbors trong ban kinh eps cho moi diem
    count_matrix = []
    for row in dist:
        pts = [en for en in row if en <= eps]
        count_matrix.append(len(pts) - 1)  # tru chinh no

    point_type = []
    for pts in count_matrix:
        is_core = "NOT"
        if pts >= min_samples:
            is_core = "core"
        point_type.append(is_core)
```

```
    core_indices = [i for i, t in enumerate(point_type) if t == "core"]

    for idx in range(len(point_type)):
        if point_type[idx] != "core":
            if any(dist[idx][j] <= eps for j in core_indices):
                point_type[idx] = "border"
            else:
                point_type[idx] = "noise"

    return point_type


result = dbscan_manual(distance, eps=EPS, min_samples=MinPts)

pd.Series(result).value_counts()
```

Out[8]:
```
core      265
border     32
noise       3
Name: count, dtype: int64
```

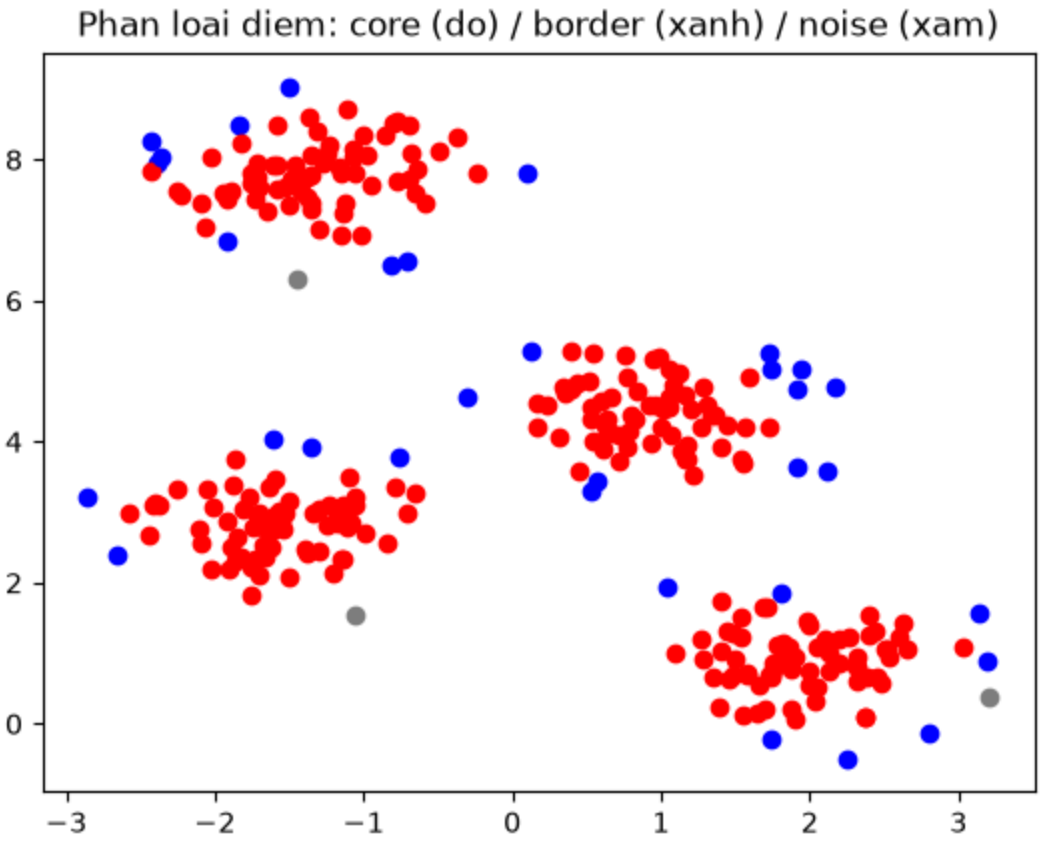
Trực quan hoá core / border / noise

In [9]:
```
scheme = {"core": "red", "border": "blue", "noise": "grey"}

for idx in range(len(result)):
    plt.scatter(X[idx][0], X[idx][1], c=scheme[result[idx]])

plt.title("Phan loai diem: core (do) / border (xanh) / noise (xam)")
plt.show()

set(result)
```



Out[9]: {'border', 'core', 'noise'}

Bước 2: Gán cluster bằng DFS (loang từ các core point)

- Bắt đầu từ một core point chưa thăm -> tạo cluster mới -> loang (DFS bằng stack) sang các điểm lân cận trong bán kính `eps`.
- Nếu hàng xóm cũng là core -> tiếp tục loang từ đó.
- Nếu là border -> gán vào cluster nhưng không loang tiếp.
- Noise -> không thuộc cluster nào, giữ giá trị -1.

Lỗi đã sửa: bản gốc có `n = 3` (hard-code, chỉ xử lý 3 điểm đầu). Đã sửa thành `n = len(point_type)` để xử lý toàn bộ 300 điểm.

In [10]:
```
def dbscan_cluster(dist, point_type, eps):
    n = len(point_type)
    cluster_id = np.full(n, -1)
    visited = np.full(n, False)
    current_cluster = 0

    for idx in range(n):
        if visited[idx] or point_type[idx] != "core":
            continue

        # Bat dau cluster moi, loang bang stack (DFS)
        stack = [idx]
```

```
        while stack:
            point = stack.pop()
            if visited[point]:
                continue
            visited[point] = True
            cluster_id[point] = current_cluster

            if point_type[point] == "core":
                neighbors = np.where(dist[point] <= eps)[0]
                for nb in neighbors:
                    if not visited[nb]:
                        stack.append(nb)

        current_cluster += 1

    return cluster_id


cluster_id = dbscan_cluster(distance.values, result, eps=EPS)
print(cluster_id)
print()
print("So cluster tim duoc:", len(set(cluster_id)) - (1 if -1 in cluster_id
pd.Series(cluster_id).value_counts()
```

```
[ 2  0  1  0  2 -1  3  1  0  0  3  0  1  0  2  1  1  2  3  3  2  2  1  3
  3  1  2  1  3  1  0  0  1  0  0  0  0  0  3  2  1  3  1  1  3  3  0  3
  0  2  3  2  0  2  2  3  0  3  0  2  0  1  0  3  3  3  0  2  0  3  1  3
  0  3  3  0  3  1  2  0  2  1  2  2  0  1  2  1  0  0  1  2  0  3  3  1
  2  2  1  3  0  2  0  2  1  2  2  1  0  1  3  3  2  0  2  1  0  2  2  1
  3  2  3  2  2  2  2  3  2  3  0  3  3  2  0  3  3  0  1  0  0  3  1  3
  1  3  0  1  0  0  0  1  0  1  2  3  0  3  2  1  0  1  1  2  1  3 -1  1
  2  1  1  0  2  1  3  0  2  2  1  3  2  1  3  3  1  1  1  1  2  0  1  3
  1  1  3  3  3  1  3  0  1  3  2  3  1 -1  3  0  1  0  1  3  1  1  0  3
  3  2  2  1  0  2  2  3  2  3  1  0  0  1  1  0  1  2  3  1  2  3  0  3
  2  1  2  0  0  0  0  3  3  0  1  3  2  1  3  3  3  2  2  0  1  1  3  2
  0  3  1  0  1  2  2  3  3  1  2  2  2  1  0  0  2  2  1  2  2  2  0  3
  0  1  2  2  0  0  0  2  2  1  0  3]

So cluster tim duoc: 4
```

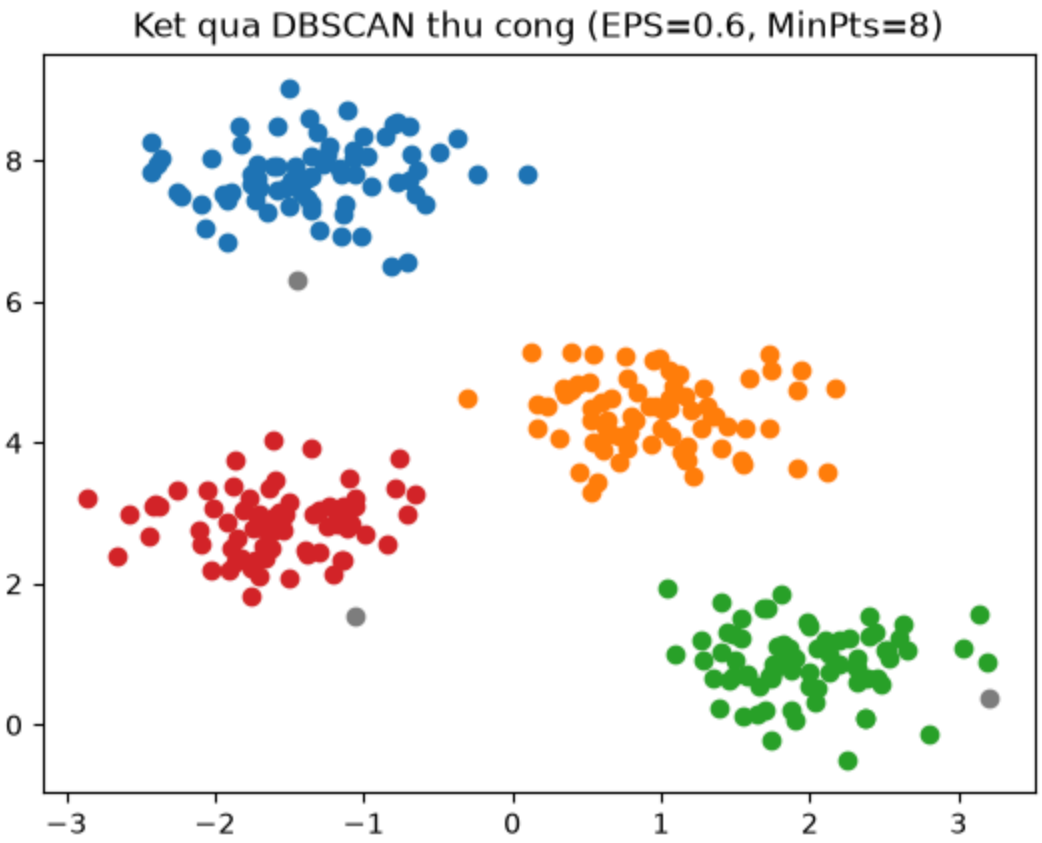
Out[10]:
```
 1    75
 2    74
 0    74
 3    74
-1     3
Name: count, dtype: int64
```

Trực quan hoá kết quả cluster cuối cùng

In [11]:
```
palette = plt.cm.tab10.colors  # bang mau cho cac cluster

for idx in range(len(cluster_id)):
    cid = cluster_id[idx]
    color = "grey" if cid == -1 else palette[cid % len(palette)]
    plt.scatter(X[idx][0], X[idx][1], c=[color])
```

```
plt.title(f"Ket qua DBSCAN thu cong (EPS={EPS}, MinPts={MinPts})")
plt.show()
```



(Phụ lục) Dò thử EPS / MinPts

Đoạn dưới đây chạy thử nhiều tổ hợp `eps`, `min_samples` để xem tổ hợp nào ra đúng 4 cụm với ít noise nhất — cách này giúp chọn tham số một cách có căn cứ thay vì đoán mò.

In [12]:
```
for eps_try in [0.4, 0.45, 0.5, 0.6, 0.7]:
    for minpts_try in [5, 8, 10]:
        res_try = dbscan_manual(distance, eps=eps_try, min_samples=minpts_tr
        cid_try = dbscan_cluster(distance.values, res_try, eps=eps_try)
        n_clusters = len(set(cid_try)) - (1 if -1 in cid_try else 0)
        n_noise = list(cid_try).count(-1)
        print(f"eps={eps_try:<4} minpts={minpts_try:<3} -> clusters={n_clust
```

```
eps=0.4  minpts=5   -> clusters=4, noise=23
eps=0.4  minpts=8   -> clusters=4, noise=46
eps=0.4  minpts=10  -> clusters=4, noise=61
eps=0.45 minpts=5   -> clusters=4, noise=16
eps=0.45 minpts=8   -> clusters=4, noise=29
eps=0.45 minpts=10  -> clusters=4, noise=36
eps=0.5  minpts=5   -> clusters=4, noise=9
eps=0.5  minpts=8   -> clusters=4, noise=22
eps=0.5  minpts=10  -> clusters=4, noise=25
eps=0.6  minpts=5   -> clusters=4, noise=3
eps=0.6  minpts=8   -> clusters=4, noise=3
eps=0.6  minpts=10  -> clusters=4, noise=5
eps=0.7  minpts=5   -> clusters=4, noise=0
eps=0.7  minpts=8   -> clusters=4, noise=0
eps=0.7  minpts=10  -> clusters=4, noise=1
```

(Tuỳ chọn) Chọn ngẫu nhiên vài điểm để minh hoạ trên biểu đồ

In [13]:
```
rng = np.random.default_rng()
numbers = rng.integers(low=0, high=len(X) - 1, size=4)
random_pts = np.array([X[n] for n in numbers])

for idx in range(len(result)):
    plt.scatter(X[idx][0], X[idx][1], c=scheme[result[idx]])
plt.scatter(random_pts[:, 0], random_pts[:, 1], c="green", s=100, edgecolors
plt.legend()
plt.show()
```

